

Java Exception Handling – Learning Guide

1. What is an Exception?

An exception is an event that disrupts the normal flow of program execution. Exception handling allows a program to respond to exceptional situations.

2. try and catch

Code that may produce an exception can be placed inside a try block. A catch block handles a matching exception.

Example: `try { int x = 10 / 0; } catch (ArithmeticException e) { System.out.println("Cannot divide by zero"); }`

3. finally

The finally block is used for code that should normally execute after try and catch processing, such as cleanup operations.

4. throw

The throw keyword is used to explicitly throw an exception from a program.

Example: `throw new IllegalArgumentException("Invalid value");`

5. throws

The throws keyword declares that a method may pass specified exceptions to its caller.

Example: `void readFile() throws IOException { }`

6. Checked and Unchecked Exceptions

Checked exceptions are generally checked by the compiler and must be handled or declared. `RuntimeException` and its subclasses are unchecked exceptions.

7. Multiple Catch Blocks

A try block can be followed by multiple catch blocks for handling different exception types. More specific exception types should be handled before broader types.

8. Quick Review

try contains risky code, catch handles exceptions, finally is used for cleanup, throw explicitly creates an exception, and throws declares possible exceptions.